Connections AI Solver

Studienarbeit

for the Informatik Department at the

DHBW Heidenheim

by

Ziyi Hong

TODO. TODO 2026

Processing period	January – March 2026, July – September 2026
Student ID	2694933
Course	Allgemeine Informatik
Evaluator	Tobias Heß

Abstract

Contents

CoT Chain-of-Thought

CSP Constraint Satisfaction Problem

List of Figures

List of Tables

1 Introduction

NYT Connections asks players to sort sixteen words into four groups of four. Each group carries one of four difficulty tiers, ranging from yellow (easiest) to purple (hardest). Published daily, the puzzle has become a popular benchmark for testing the reasoning abilities of language models. Two challenges make Connections difficult to solve.

The first challenge is lexical or semantical. Solving Connections requires several distinct kinds of language understanding at once. Some groups rest on straightforward synonymy. Others depend on shared themes, wordplay, or knowledge of pop culture and named entities. For the taxonomy defined in this thesis, see Table 3.1. A single puzzle typically mixes several of these relation types, and is deliberately constructed so that some words could plausibly belong to more than one candidate group, an effect commonly referred to as a red herring.

The second challenge is structural. Because a word cannot be reused across groups, correctly identifying even one group depends not only on recognizing a plausible shared category among four words in isolation, but also on the arrangement of the remaining words. A locally plausible grouping can still be wrong if it leaves the remaining words unable to form additional valid groups of four.

In previous studies, the first challenge has been widely discussed and mainly through embedding-based clustering or large language models to propose or judge candidate groupings (see Section 2.1).

This thesis take the same idea but different apporach. Instead of using LLM to directly suggest groups, we try several forms of knowledge injection from different sources (see Chapter 4), and utilize a scoring function to suggest candidate groups. The results find that knowledge injection consistently fall short in similar ways (note: to be written, do the reference later!!!). The second challenge, ensuring that all sixteen words are partitioned exclusively, has received comparatively little attention on its own. Taking inspiration from solving Wordle algorithmically with information theory [1], this thesis treats the puzzle's interactive mechanism: submitting a guess, observing feedback, and revising, as central to addressing this second challenge, and asks how much of the puzzle's difficulty it can resolve by algorithm.

1.1 Structure

The structure of this thesis is as follows. Chapter 2 gives an overview of the prior work relevant to solving Connections. Chapter 4 describes the proposed methodology for solving Connections.

2 Related Work

This chapter provides an overview of prior work relevant to this thesis, including work that directly solves Connections using LLMs and work that inspired the methods used in this study. It first introduces prior work on solving Connections with AI. It then presents related work on the methods used, which serves as the knowledge base for this study.

2.1 Prior Work on Connections Solving

Current work on this topic mainly uses LLMs as the main solution for solving Connections. These studies compare how well different models solve Connections. Some use Connections as a benchmark to test LLM capability, while others explore different prompting techniques and methods. Todd et al. use three methods, namely a sentence-embedding baseline, a comparison between two LLMs (GPT-3.5-Turbo-1106 and GPT-4-1106-Preview), and Chain-of-Thought (CoT) prompting. Their experiments allow five incorrect and five invalid guesses per puzzle, which departs from the original game's rules. GPT-4 Turbo with CoT achieves the highest success rate at 58.11%. The best-performing sentence-embedding baseline (MPNet) reaches 11.6%. LLMs are often stumped by categories that involve non-semantic word properties, abstract features, or context-dependent usage. They also report a finding relevant to this thesis. Puzzle order matters, since presenting puzzles in their original, grouped order achieves substantially higher accuracy than presenting them in randomized order [2].

A similar work from Samadarshi et al. benchmarked several LLMs against both novice and expert human players and creates a taxonomy of knowledge types required to solve connections. They used in total 438 Connections puzzles and their best performing models Claude Sonnet 3.5 achieved 18% success rate of fully solving the puzzles. They categorizes NYT Connections categories along two main dimensions, word form (covering morphology, orthography, phonology, and multiword expressions) and word meaning (covering semantic relations such as synonymy, polysemy, and hypernymy, along with associative and encyclopedic relations), with a further category for groups that combine both form-based and meaning-based cues. Results show that models performed comparably well on semantic relation categories but significantly worse on the other types.

More recent work from Loredo Lopez, McDonald, and Emami compare three game configurations: single attempt, four trys without feedback (one away prompt) and four trys with feedback. Human solvers outperform LLM under all three configurations. The best performing model Claude 3.5 under full hints reaches 40.25%, while the human full-hints score reaches 60.67%. They also experiment on different prompting techniques. CoT and self-consistency prompting yield diminishing or even negative returns as puzzle difficulty increases. Simple input-output prompting sometimes outperforms these more elaborate reasoning strategies. Their herustic (non-LLM) baseline, using Multilingual-E5-Large-Instruct embedding and beam search, reaches 13.25% accuracy.

Zhang, Jiang, and Ye compare in-context prompting of a large model (DBRX) against fine-tuning and distilling a smaller model (Mistral 7B) on solved puzzles. They found that fine-tuning a smaller model (Mistral 7B) directly on solved puzzles fails to capture the task itself, which underperformed even their clustering baseline. Their strongest result comes from Verbal Reinforcement Learning, in which a larger supervisor model critiques the fine-tuned model's output in natural language across three rounds, roughly doubling the average per-puzzle match rate relative to single-pass prompting [5].

2.2 Word Puzzles as Weighted Constraint Satisfaction Problem (CSP)

CSPs are mathematical questions defined as a set of objects whose state must satisfy a number of constraints or limitations [6]. A (finite domain) CSP can be expressed in the following form: given a set of variables, together with a finite set of possible values that can be assigned to each variable, and a list of constraints, find values of the variables that satisfy every constraint [7]. It is formally defined as a triple $\langle X, D, C \rangle$ where X is a set of variables, D specifies a domain of possible values for each variable, and C is a set of constraints restricting which combinations of value assignments are permitted [6].

In artificial intelligence, a Weighted Constraint Satisfaction Problem, also known as Valued Constraint Satisfaction Problem, is a constraint satisfaction problem in which preferences among solutions can be expressed [8]. A plain CSP, treats every solution as equally good: it either satisfies the constraints or it does not. Connections needs more than that. Not every four-groups-of-four split is a correct solution, only one is. What is available

instead is a measure of how well four words seem to belong together, based on their similarity. Thus, Connections can be defined as a weighted CSP. It keeps the hard constraint (a valid four-groups-of-four partition) but adds a score to each candidate value assignment, so that solving the problem means finding the constraint-satisfying assignment with the best total score, not just any assignment that satisfies the constraints.

2.2.1 Solving Word Puzzles as Weighted CSP

Applying this concept to word puzzles is not new. Ginsberg formalizes American-style crossword solving as a weighted CSP in the program Dr.Fill: each grid cell is a variable, each candidate fill is a value, and the constraints require intersecting words to agree on shared letters [9]. Because many fills are plausible for a given clue, each candidate fill is given a likelihood-based weight, and Dr.Fill searches for the assignment that maximizes total weight while still satisfying the grid's hard constraints. It draws candidate fills from several sources: a database of over 47,000 previously published crosswords with 1.9 million unique clues, two dictionaries of varying size, 1.2 million synonyms drawn from WordNet and other online sources, and a database of Wikipedia titles and word pairs. Each candidate fill is then scored on five criteria: whether the clue matches one seen before in the clue database, whether the fill's part of speech agrees with the part of speech inferred from the clue, the fill's crossword "merit" (a hand-rated measure of how desirable a word is as a crossword answer in general), whether the fill is an abbreviation, and whether the clue is a fill-in-the-blank type resolvable against Wikipedia text. Dr.Fill then searches for the assignment that maximizes total score while still satisfying the grid's hard letter-intersection constraints. Two further design choices in Dr.Fill are worth noting. First, they found that looking further ahead before committing to a choice gave no real benefit, and the system used only a single step of lookahead instead. Second, branch-and-bound, a standard pruning technique for weighted CSPs, was tested and dropped because it added little value in this domain, as a result postprocessing was used instead. In both cases, a more sophisticated search method did not beat a simpler one, once the scoring function and hard constraints were doing most of the work. Since complete knowledge of what each clue means is impractical to obtain, Dr.Fill relies mainly on the hard constraint that crossing words must share letters to narrow down candidates, not on understanding the clues themselves. Dr.Fill inspired the methodology used in this thesis, which also

treats Connections as a weighted CSP and uses a scoring function to evaluate candidate groupings.

2.2.2 Set Partitioning

The constraint C of Connections (every word belongs to exactly one group and every group contains exactly four words), more precisely speaking, is a combinatorial optimization problem known as set partitioning. A partition of a set is a way of dividing all of its elements into non-empty, pairwise disjoint subsets whose union is the original set [10]. The set partitioning problem asks, given a ground set and a collection of candidate subsets each associated with a cost, to select a subset of these candidates that forms a partition of the ground set at minimum or maximum total cost [11]. The sixteen Connections puzzle words are the ground set, candidate four-word groups are subsets among which a selection must be made, and each candidate group's score from the weighted CSP explained above serving is its cost. Identifying this structure is what motivates the grouping strategy adopted later in this thesis.

Formulating a puzzle as a weighted CSP defines what to search for. Among all groupings that satisfy the hard constraint, the goal is the one with the highest total score. This does not guarantee that the grouping is correct. The hard constraint only rules out invalid partitions. Many valid partitions remain, and the scoring function is what distinguishes them from each other. If the scoring function is imperfect, the highest-scoring grouping can still be wrong. Puzzle games like Connections give players a further source of information, namely the response the game returns after each attempt. Using this response turns the problem from a single optimization into a sequence of attempts, where each attempt narrows down what remains possible. Section 2.3 covers how much a single response is worth, and Section 2.4 covers what to do with it.

2.3 Information Theory

Information theory is the mathematical study of the quantification, storage, and communication of a particular type of mathematically defined information [12]. It gives information a precise mathematical definition grounded in probability [13]. Gaining information and reducing uncertainty about a random event are the same thing. By convention, information is measured in base-2 logarithms, giving the bit as its unit: each time half of the

probability mass is ruled out, exactly one bit is gained [14]. The information contained in an outcome with probability $p(x)$ is $\log_2 \frac{1}{p(x)}$.

The probability-weighted average of this information across all outcomes is the entropy, denoted $H(\mathrm{X})$:

$$H(\mathrm{X}) = \sum_{x \in \mathcal{X}} p(x) \log_2 \frac{1}{p(x)}. \tag{2.1}$$

Entropy serves as the connection between probability and information content. It measures how uncertain a random event is on average. Higher entropy means more uncertainty to begin with, so more information must be acquired to become certain about the outcome. Entropy is maximized when probability is spread evenly over all outcomes. In that case it reduces to

$$H_{\max}(\mathcal{X}) = \log_2 |\mathcal{X}|, \tag{2.2}$$

where $|\mathcal{X}|$ is the number of possible outcomes [14].

Conditional entropy, $H(\mathrm{X} \mid y)$, measures how much uncertainty about X remains after observing an outcome y. The reduction in entropy achieved by an observation is the information gain. Taken in expectation over all possible outcomes of an observation, before that observation is actually made, this gives the expected information gain of an action:

$$IG(\text{guess}) = H(\mathrm{X}) - \mathbb{E}\big[H(\mathrm{X} \mid \text{feedback})\big]. \tag{2.3}$$

This quantity is what solvers use to decide which action is most useful before knowing what the feedback will be. An action with higher expected information gain is expected to narrow down the remaining candidates more, regardless of which specific feedback is eventually returned.

2.3.1 Solving Wordle with Information Theory

Wordle is another word puzzle game from NY Times. It asks the player to guess a five-letter word within six attempts. After each guess, every letter is marked as correct and in the right position, present but in the wrong position, or absent, giving detailed per-letter feedback. Information theory based Wordle solvers apply this framework directly: at each step, the candidate word maximizing expected information gain is chosen as the next guess [15]. According to Grant Sanderson pointed out in his video, the best possible first guess yields an expected information gain of 5.89 bits, reducing the remaining uncertainty by

roughly half in a single observation [1]. Aladaileh et al. report that entropy-based selection outperforms a heuristic that selects words by letter distribution, and propose the approach as a general baseline for applying Shannon entropy to other games [15]. Information theory can be applied to Connections as well. Detailed description see Chapter 3.

2.4 Strategies for Puzzle Games

Section 2.2 introduces the structural constraint, and then Section 2.3 as a way of measuring how much a single observation can reveal. After measuring information, a solver needs a rule that turns the current state of knowledge into the next move. This section covers the other question: given feedback from previous attempts, how should the next guess be chosen?

Cunanan and Thielscher provide a solution for Wordle and for a wider class of games with the same structure [16]. After a sequence of guesses and responses, the candidate set is the set of possible secrets that remain consistent with every response observed so far. A strategy is a rule that maps the current candidate set to the next guess, and different strategies can be compared by scoring every possible guess against the candidate set.

Several such scoring functions were developed earlier for Mastermind. These functions include minimizing the size of the largest candidate set that could remain, maximizing information gain as described in the previous section, and maximizing the number of distinct responses a guess can produce [17]. Cunanan and Thielscher test these functions on Wordle and find that none performs best on its own. Maximizing information gain in particular does not yield the optimal strategy [16].

3 Problem Formation

Chapter 2 introduced three foundational concepts that apply to Connections. Firstly, it can be treated as weighted CSP and its grouping requirement can be concluded as a set partition problem. Secondly, information theory provide a framework to measure how much remains unknown about a puzzle and how much a single observation can reveal. Thirdly, after quantifying the information and uncertainty, connections provides a feedback mechanism that can be used to determine the next answer.

The purpose of this chapter is to explain the rules and mechanics of Connections using the mathematical terminology introduced above. Section 3.1 states the Connections game rules, provides a taxonomy of the relations that can exists, and defines the two evaluation settings used throughout this thesis. Section 3.2 expresses the puzzle as a weighted CSP mathematically. Section 3.3 counts how many candidate solutions exist and how much uncertainty that represents. Section 3.4 then quantifies how much of that uncertainty the game's feedback can remove.

3.1 Problem Setting

A Connections puzzle presents sixteen words that must be split into four groups of four. Each group corresponds to a hidden category, and every word belongs to exactly one group. The player does not submit a full solution at once. Instead, one group of four words is submitted at a time, and the game responds before the next submission is made.

Three responses are possible. If all four submitted words indeed belong to one category, 4 words will be placed on the same row and the category will be revealed. If exactly three of them do, player will be prompted with one away. Otherwise the response is wrong. Both one-away and wrong count as mistakes, and the game ends in failure after four mistakes. Correct submissions do not consume the mistake budget. A correct submission takes those four words out of consideration for the remaining groups. The puzzle therefore shrinks as it is solved, and each correct answer both reduces the number of remaining possibilities and simplifies the problem that remains.

There are four levels of difficulty, starting with yellow group (the easiest), then green, blue, and purple (the hardest), which reflect the intended difficulty in recognizing the

Table 3.1: Relation category for NYT Connections with illustrative examples.

Relation type	**Example category + words**
Semantic similarity	*FURTHEST POINT*: END, EXTREME, OPPOSITE, POLE [19]
Semantic relatedness	*THINGS YOU CAN DO ON SOCIAL MEDIA*: COMMENT, LIKE, LURK, POST [19]
Form- or rule-based relations	*ART MOVEMENTS, WITH "-ISM"*: BRUTAL, IMPRESSION, MANNER, REAL [19]
Referential relations	*RIHANNA #1 HITS*: DIAMONDS, SOS, UMBRELLA, WORK [20]
Mixed categories	*CAR BRAND HOMOPHONES*: INFINITY, MINNIE, OPAL, OUTIE [21]

underlying grouping relation. So if player are able to solve the easy groups first, the remaining puzzle will be easier to solve.

The group relations are diverse. Some categories are driven by semantic similarity, such as near-synonymy or membership under a shared hypernym (e.g., types of flowers). Others reflect semantic relatedness: words are not strictly similar, but cohere within a common theme or script (e.g., actions in a typical scenario). A third class consists of form- / rule-based relations, where membership is determined by surface regularities such as spelling or word-formation patterns (orthography, morphology), or by conventional meanings of symbols and abbreviations. Referential relations require external world knowledge about named entities or factual lists (e.g., trivia- or culture-dependent categories).

Notably, the hardest (purple) categories often draw on less standard relations such as word and letter patterns (e.g., rhymes, homophones, anagrams, palindromes, or unmarked fill-in-the-blank phrases) and may also involve pop-culture or proper-noun knowledge. [18] As a result, they frequently exhibit mixed relations that combine more than one relation type and cannot be explained well by a single type alone.

An overview of these relation types and representative examples is provided in Table 3.1.

This categorisation is derived from inspecting published puzzles, and it broadly aligns with the taxonomy proposed by Samadarshi et al., which concentrates on the kinds of knowledge a LLM must possess [3], while ours classifies the relation that holds a group together. The single semantic category they proposed was split into similarity and relatedness in this taxonomy, since the two behave differently under embedding-based scoring, and multiword expressions was also nol longer treated as an individual category.

3.1.1 Evaluation Settings

This thesis evaluates solvers under two settings: one-shot and interactive. In the one-shot setting, a solver commits to a complete partition of all sixteen words in a single pass, receiving no feedback at any point. The same setting is also used by Todd et al. [2], Samadarshi et al. [3], and Loredo Lopez, McDonald, and Emami [4]. In the interactive setting, the solver submits one group at a time, observes the response, and may revise before submitting again, subject to the four-mistake budget, which is also adopted by Loredo Lopez, McDonald, and Emami [4]. The second reflects how the puzzle is actually played.

The one-shot setting isolates what is being measured. With no feedback available, the outcome depends entirely on the scoring function, so any change in performance can be attributed to whatever was changed in that function. Under the interactive setting the two are entangled: a weak scoring function may be rescued by feedback, and a strong one may be squandered by a poor choice of what to submit first, leaving it unclear which is responsible for the result. The interactive setting measures whether a puzzle is won under the rules. The two settings pose different problems and are measured accordingly. The metrics used for each are given in Chapter 4.

3.2 Connections as a Weighted CSP

The previous section described the puzzle in words. This section restates it in the form introduced in Section 2.2, where a constraint satisfaction problem is given as a triple $\langle X, D, C \rangle$ of variables, domains and constraints, and a weighted variant adds a score to each candidate assignment so that solutions can be ranked rather than merely accepted or rejected. Both parts are needed here. The constraint captures what makes a partition legal, and the score captures what makes one legal partition more plausible than

Figure 3.1: The NYT Connections puzzle answer of 12 June 2023.

another.

Connections maps onto the triple $\langle X, D, C \rangle$ directly. Let $W = \{w_1, \ldots, w_{16}\}$ be the sixteen puzzle words. Each word gets its own variable, so $X = \{x_1, \ldots, x_{16}\}$. The variable x_i does not stand for the word itself, but for the decision of which group w_i is placed in. Every variable has the same domain, $D_i = \{1, 2, 3, 4\}$, one value for each of the four groups.

Take the puzzle of 12 June 2023 as an example (Figure 3.1 shows the answer). Its sixteen words are BUCKS, HAIL, HEAT, JAZZ, KAYAK, LEVEL, MOM, NETS, OPTION, RACECAR, RAIN, RETURN, SHIFT, SLEET, SNOW and TAB. Numbering its sixteen words gives $w_1 =$ BUCKS, $w_2 =$ HAIL, $w_3 =$ HEAT, and so on up to $w_{16} =$ TAB. Placing BUCKS in group 2 means setting $x_1 = 2$, and placing JAZZ there as well means $x_4 = 2$. A complete assignment $x = (x_1, \ldots, x_{16})$ records one such decision per word and therefore describes one way of dividing the board. One important aspect is that the group numbers themselves carry no special meaning for the solver in this thesis. They serve only to record which words end up together, and any consistent relabelling describes the same division of the board. A solver is not asked which group corresponds to which, nor to identify the

difficulty level a group belongs to, only which four words go together.

Let k be a group number, so $k \in \{1, 2, 3, 4\}$, and write G_k for the set of words that have been assigned to it:

$$G_k = \{w_i : x_i = k\}. \tag{3.1}$$

The condition $x_i = k$ selects those indices i whose word was placed in group k, and G_k collects the corresponding words. In the assignment described above, $x_1 = 2$ and $x_3 = 2$, so BUCKS and HEAT both belong to G_2. Every assignment x determines all four sets $G_1, \ldots, G_4$ in this way. The two notations describe the same thing from different directions: x records, for each word, which group it went to, while G_k records, for each group, which words it received.

The constraint set C then requires each group to hold exactly four words:

$$C: \quad |G_k| = 4 \quad \text{for every } k \in \{1, 2, 3, 4\}. \tag{3.2}$$

Satisfying Equation 3.2 is not difficult, but only one partition recovers the puzzle's intended grouping. The constraint treats all valid partitions as equally acceptable and we need something to distinguish the correct one, and that role is filled by a scoring function.

Let $s(g)$ be a function that takes a group of four words $g \subset W$ and returns a score or weight for how plausibly they belong to a common category, with higher values indicating stronger evidence. How s is constructed in detail is explained in Chapter 4. The problem is then to find

$$x^* = \arg\max_x \sum_{k=1}^{4} s(G_k) \quad \text{subject to } |G_k| = 4 \text{ for all } k. \tag{3.3}$$

The assignment x^* is the one scoring highest among all assignments satisfying the constraint. It is worth being explicit that x^* is not by definition the correct answer, but the partition that s ranks highest, and whether that coincides with the puzzle's intended grouping depends entirely on how well s reflects the relations the puzzle is built on. Where s is imperfect, the correct partition may score below an incorrect one, and no amount of searching will recover it, because the information needed to prefer it was never encoded in the score. Much of this thesis is concerned with how close s can be brought to that standard, and with what happens when it falls short.

The sum in Equation 3.3 runs over all four groups, so an assignment is judged by the combined plausibility of the whole partition and not of any group on its own. The 12 June 2023 puzzle shows why this matters. HEAT sits naturally alongside HAIL, RAIN, SLEET and SNOW, and a solver scoring that group in isolation would find it convincing. The group is nonetheless wrong: HEAT belongs to NBA TEAMS, and committing to a weather group containing it leaves BUCKS, JAZZ and NETS without a fourth member. A locally plausible group can only be accepted if the twelve words it leaves behind can still form three groups that score well, and the summation over all four groups is what enforces that condition.

3.3 The Hypothesis Space

In Equation 3.3, the solving was defined as a search over assignments satisfying the constraint. In this section, we quantify the size of the search space. How large that search is depends on how many such assignments exist. $\mathcal{H}$ is used to represent the set of all valid partitions of the sixteen words into four groups of four, so that solving the puzzle is equivalent to selecting one element of $\mathcal{H}$.

The size of $\mathcal{H}$ is calculated using permutations and combinations. Arranging the sixteen words in a sequence can be done in 16! ways, and each arrangement can be read as a partition by taking the first four words as one group, the next four as the second, and so on. But there are two overcounting factors we need to divide out. First, the order of words within a group carries no information, since a group is a set. HAIL, RAIN, SLEET, SNOW and SNOW, SLEET, RAIN, HAIL are the same group, and each group can be internally ordered in 4! ways, giving a factor of $(4!)^4$ across the four groups. Second, the order of the groups themselves carries no information either, as established in Section 3.2, so the 4! ways of arranging the four groups must also be divided out. What remains is

$$|\mathcal{H}| = \frac{16!}{(4!)^4 \cdot 4!} = 2{,}627{,}625. \tag{3.4}$$

A solver that knows nothing about a particular puzzle has no reason to favour any element of $\mathcal{H}$ over another. Applying the entropy formula Equation 2.2 to a uniform distribution over $\mathcal{H}$ and entropy is at its largest in this case. It gives

$$H_{\max}(\mathcal{H}) = \log_2 |\mathcal{H}| = \log_2(2{,}627{,}625) \approx 21.3 \text{ bits}. \tag{3.5}$$

This is the total amount of information required to identify the correct partition with certainty, starting from no knowledge at all. The information a solver gets, no matter it's from the feedback or encoded in the scoring function, needs to reach this figure before it can be guaranteed to find the correct answer.

3.4 Information from Feedback

After knowing how much information is needed to solve a puzzle, we now turn to how much information the game mechanism actually provides. Under the interactive setting, each submission returns one of three responses: correct, one away, and when it shows nothings then means it is wrong. It depends on how many words the submitted group shares with whichever correct group it overlaps most. Four shared words returns correct, three returns one away, and anything less returns wrong. The last case can mean that a submission overlapping a correct group in two words or the 4 submitted words separately belong to 4 individual groups.

The response set has three elements (correct, one away, wrong), so by Equation 2.2 a single response carries at most

$$\log_2 3 \approx 1.58 \text{ bits.} \tag{3.6}$$

This is the maximum amount of information that can be obtained from a single feedback response, regardless of the submission strategy and is attained only when all three are equally likely. So it is the ideally maximum information gain. In real Connections, the distribution of feedback outcomes is not uniform. Of the $\binom{16}{4} = 1820$ groups that can be submitted, four are correct and 192 are one away, leaving 1624 that return wrong. A submission chosen at random therefore returns wrong with probability $1624/1820 \approx 0.89$, and the entropy of the response under that distribution is about 0.55 bits, roughly a third of the ceiling.

The game allows four mistakes. Since correct submissions do not consume the budget, the number of responses received over a game is not fixed, but the number of failures is restricted, which is at most four responses of one away or wrong are available before the game ends. The total maximum information obtainable from feedback is therefore

$$4 \log_2 3 \approx 6.34 \text{ bits.} \tag{3.7}$$

Equation 3.5 gives the information required to identify the correct partition as roughly 21.3 bits. Feedback supplies at most 6.34 of them and the remaining 15 bits must come from somewhere else. In this thesis, it is the scoring function that we mentioned in Section 3.2, which will be introduced in Chapter 4.

4 Methodology

Approaches to solving Connections fall into two broad groups. The first prompts a language model to produce the four groups directly, which is the primary solution used in prior work Section 2.1. The second scores candidate groups and then searches for the partition whose groups score highest using scoring function and searching algorithm, which is served mainly as a baseline in the prior work. The hypothesis space articulated in Section 3.3 applies to both. Most of the information needed to identify the correct partition has to come from the solving mechanism rather than from the game feedback. The two approaches differ in where that information sits. In the first it is held in the model's parameters and reached through the prompt. In the second it is held in the scoring function and the search algorithm, which solves Connections by searching for the partition with the highest score subject to the grouping constraint (described in Section 3.2).

This thesis concentrates on the second approach. A scoring function can be taken apart into components that are swapped and evaluated one at a time, so the contribution of each component to the final result can be measured directly. A language model is however a black box. A change in the prompt can change the output, but it is hard to directly observe and explain what exactly happened.

The scoring function assigns a score to every partition, and works generally in two steps. A similarity is first computed for every pair of the sixteen words. These similarities are then assembled into scores for candidate groups, and from those into scores for partitions. Under the one-shot setting the partition with the highest score is returned as the answer. Under the interactive setting the scores are instead treated as a prior over all partitions and revised as responses come in.

Section 4.1 describes the scoring function and the choices it involves. Section 4.2 describes solving under the one-shot setting. Section 4.3 describes what the interactive setting adds. Lastly, **??** assembles the components into a complete system.

4.1 Scoring Functions

The scoring function assigns a value to every partition, so that the best partition can be identified. It first produces a similarity for every pair of words, collected in a 16×16

matrix of $\binom{16}{2} = 120$ pairwise similarities. These similarities are then assembled into scores for groups and for partitions.

A candidate group is any four of the sixteen words, and each contains $\binom{4}{2} = 6$ word pairs. Its score is the sum of all the pairwise similarities in the group:

$$s(g) = \sum_{\{i,j\} \subset g} \text{sim}[i, j]. \tag{4.1}$$

A partition is then scored by adding the scores of its four groups, which is the quantity maximized in Equation 3.3. Since the same group appears in many partitions, all $\binom{16}{4} = 1820$ group scores are computed once and stored, and partition scores are assembled from them by lookup.

What varies is how $\text{sim}[i, j]$ is obtained, and Figure 4.1 shows the two configurations compared. The first embeds each word as it is, so that a word has one vector and the similarity between two words is the cosine similarity of those vectors. The second expands each word into a set of texts, one per meaning, and embeds each text separately, so that a word is represented by a pool of vectors. The similarity between two words is then obtained by aggregating over the most similar pairings of their meanings, as detailed in Subsection 4.1.1.

4.1.1 Semantic Enrichment

A word can carry multiple meanings. The word "bank" can refer to a financial institution or to the edge of a river, and an embedding of the bare word blends both.

The simplest way to obtain $\text{sim}[i, j]$ is to embed each word directly and take the cosine similarity of the two vectors, which is what the embedding-based baselines in Section 2.1 do, and those baselines generally perform poorly. Two kinds of embedding model are involved. Static word embeddings, such as Word2vec and GloVe, assign one fixed vector per word, so every meaning of a polysemous word is compressed into a single representation, a property known as the meaning conflation deficiency. Contextualised models, such as BERT, address this by producing a vector that depends on the surrounding text [22]. This thesis specifically uses sentence-embedding models, a family of contextualised models that pool an entire piece of text into a vector, making it possible to embed a word's definition as a whole rather than the bare word alone.

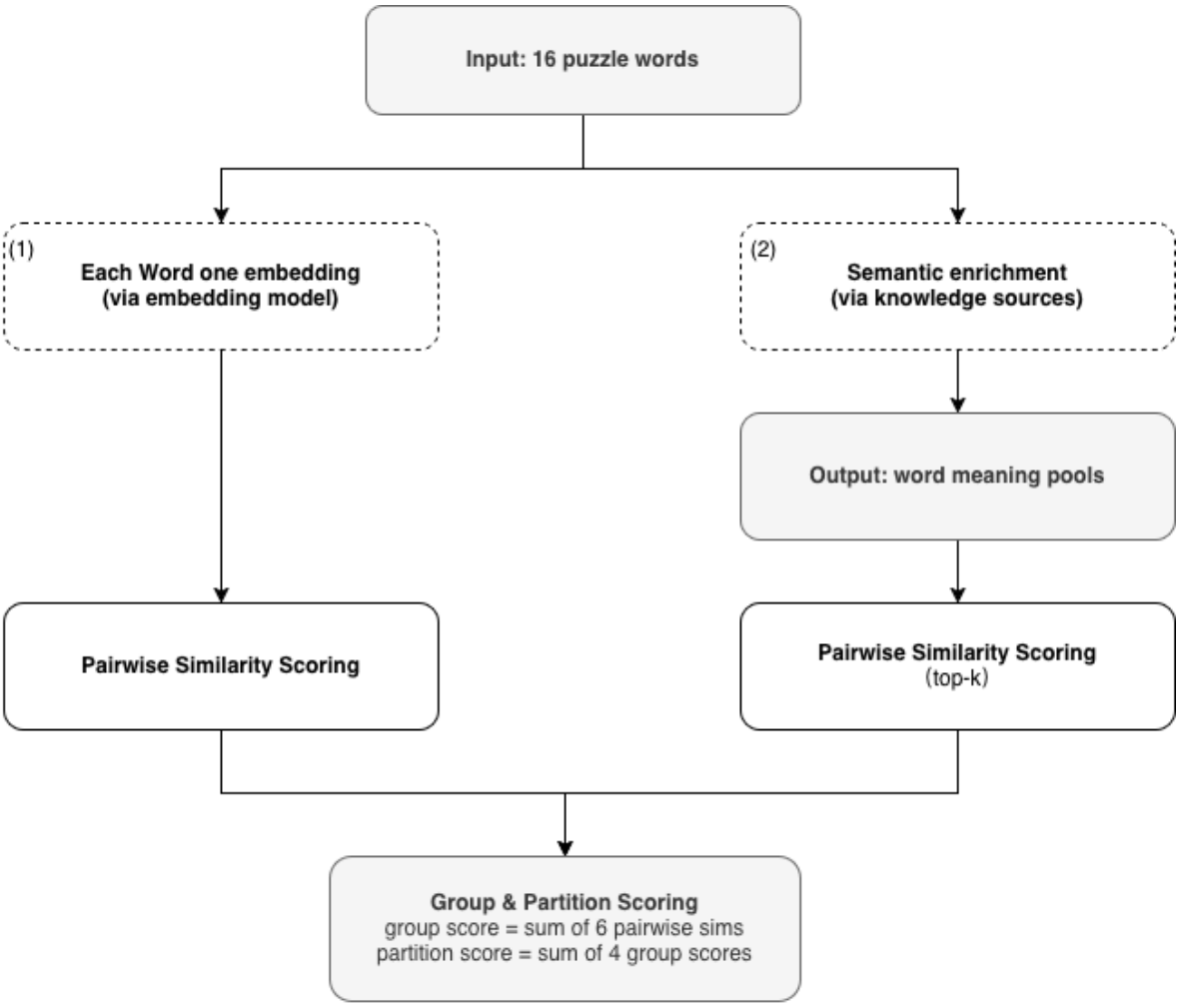

Figure 4.1: The two configurations of the scoring function compared in this thesis. Dashed borders mark the points where configurations differ; solid borders mark stages that are the same throughout. Configuration (1) embeds each word directly. Configuration (2) expands each word into a pool of meanings first, and then aggregates over the most similar pairings of meanings into the single similarity.

Connections requires the intended meaning of each word to be identified before the words can be partitioned, and the most obvious meaning is not necessarily the intended one. A sentence-embedding model is well suited to this, because the vector it produces reflects the specific text it is given, unlike a static embedding, which assigns the same vector to a word regardless of what text accompanies it. Todd et al. likewise use a sentence-embedding model as their baseline [2]. This property motivates enriching each word with its candidate meanings before embedding.

Our solver supplies every candidate meaning of a word directly as a set of texts, each describing one meaning. Each text is embedded separately with a sentence-embedding

model, giving each of a word's meanings its own vector instead of averaging them into one. This avoids the meaning conflation described above and lets the intended meaning stand out on its own. If two words are strongly similar under one particular pairing of their meanings, that pairing is itself evidence of what the puzzle intends.

The texts themselves come from external lexical and encyclopedic resources. Which resources are used is a configuration choice rather than part of the mechanism. The knowledge resources examined in this thesis and the effect of each are reported in Chapter 5. When none of the enabled resources returns anything for a word, the bare word itself is used as the single entry in its pool, so that every word is represented.

After enrichment, each word is represented by a pool of vectors rather than a single one. Comparing two words therefore produces not one similarity but many, one for every pairing of a meaning from the first word with a meaning from the second. A single value, $\mathrm{sim}[i, j]$, still has to be produced from this collection.

Both extremes raise a concern. Averaging over every pairing treats all of a word's meanings as equally relevant, even though a puzzle intends only one, which risks diluting the one match that actually matters. A word with many meanings would be diluted more than a word with few, simply because it contributes more irrelevant pairings to the average. Taking only the single highest similarity avoids this risk, but relies on one pairing alone, so a single coincidentally high similarity could decide the result with nothing to cushion it.

This thesis takes the average similarity over the top-k highest-similarity pairings. Small k behaves like taking the maximum alone. Large k approaches averaging over everything. k is a parameter that interpolates between the two extremes, not a value that can be fixed in advance.

The value of k is determined experimentally rather than fixed here. A range of values is compared, and the one that performs best is adopted, as reported in **??**.

4.2 Solving under the One-Shot Setting

Under the one-shot setting, the solver submits a single partition without receiving any feedback, so the task is to identify that one partition directly. With all $\binom{16}{4} = 1820$

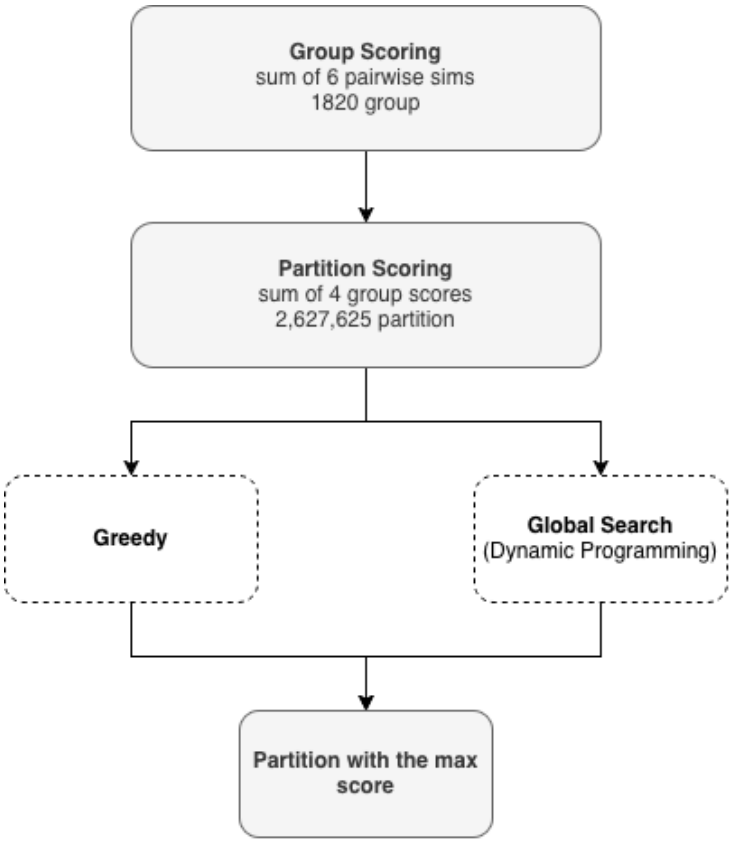

Figure 4.2: Pipeline for solving Connections under the one-shot setting.

candidate groups scored by Equation 4.1, what remains is to solve Equation 3.3 directly, searching the $|\mathcal{H}| = 2{,}627{,}625$ valid partitions (Section 3.3) for the one with the highest total score. The highest score can be pursued in two different ways. A local approach picks the highest-scoring group at each step, then repeats the process on the words that remain until a partition is formed. A global approach instead searches for the partition whose four groups have the highest score when summed together, even if no single group in it is the highest-scoring one on its own. Two search algorithms, one for each approach, are compared in this thesis, and Figure 4.2 gives an overview.

The first is greedy. The highest-scoring group among all 1820 candidates is fixed as the first group. Its four words are removed, and the highest-scoring group among the candidates formed from the remaining twelve is fixed as the second. The same is done for the remaining eight words, and the last four words form the fourth group by default. Each choice is made from what scores best at that point and is never revisited.

The greedy procedure never reconsiders an earlier choice, even if it turns out to leave the remaining words in a bad position. The second algorithm instead achieves the global

approach directly, using dynamic programming as the method that makes this possible.

Dynamic programming solves a problem by considering every possible subproblem and reusing each one's solution wherever it reappears, rather than committing to a single path through it [23]. Considering every subproblem this way is what guarantees the best score found is the global best, matching the global approach described above, while reusing solutions instead of recomputing them is what keeps this exhaustive search efficient. Connections can be solved with dynamic programming. Since a partition's score is the sum of its four group scores, the best partition of a set of words is simply the best last group plus the best partition of the words left over, so solving the full puzzle reduces to solving the same kind of problem on smaller sets of words. Because many different choices of groups lead back to the same smaller set, solving each subset once is enough, and the small number of reachable subsets keeps this cheap enough to run on every puzzle.

Recall that W denotes the sixteen puzzle words. Let M denote a subset of these words. In practice, M is represented as a 16-bit mask, an integer whose i-th bit is 1 if the i-th word is included in M and 0 otherwise, so that every subset of W corresponds to exactly one such integer. This representation is used in this thesis's implementation because it makes $\text{best}(M)$ cheap to store and look up for every subset, since M itself can serve directly as an index, and because operations such as removing a group from M reduce to fast bitwise arithmetic rather than manipulating an explicit list of words. Let $\emptyset$ denote the empty subset containing no words at all, corresponding to the mask with every bit set to 0. For any such $M \subseteq W$, let $\text{best}(M)$ denote the highest score obtainable by partitioning M alone into groups of four. When M is empty there are no words left to group, so nothing is scored, and $\text{best}(\emptyset) = 0$. This is the starting point the rest of the computation builds on.

For a non-empty M, imagine one specific group g is part of the eventual solution. The remaining words, $M \setminus g$, still need to be partitioned as well as possible. The best achievable score for them is by definition $\text{best}(M \setminus g)$. Committing to g therefore contributes a total of $s(g) + \text{best}(M \setminus g)$. It is not known in advance which group actually belongs to the best solution. Every possible g is therefore tried, and the highest resulting total is kept. This defines $\text{best}(M)$ recursively, in terms of best evaluated on strictly smaller subsets. It follows the same subset-enumeration technique used for exact dynamic programming

over subsets [24],

$$\text{best}(M) = \max_{\substack{g \subseteq M \\ |g|=4}} \Big(s(g) + \text{best}(M \setminus g) \Big). \tag{4.2}$$

This recurrence does not run in circles. Each step removes a group of four words, so $M \setminus g$ is always four words smaller than M. Every subproblem therefore eventually reduces to the base case $\text{best}(\emptyset) = 0$. This thesis evaluates Equation 4.2 bottom-up. Masks are processed in order of increasing size, starting from $\text{best}(\emptyset) = 0$. By the time $\text{best}(M)$ is computed for any given M, every smaller $\text{best}(M \setminus g)$ it needs has already been computed and stored. No value is therefore ever recomputed. The last mask processed is W itself, giving $\text{best}(W)$, the score of P^* in Equation 3.3.

The score alone is not the answer. To recover the four groups that make up P^*, whichever group g achieves the maximum in Equation 4.2 is recorded for every M along the way. Starting from W, this recorded group is read off, removed, and the same lookup is repeated on what remains, until all sixteen words have been assigned.

The two differ in what they take into account. The greedy procedure judges a group by its own score alone. The recursion judges it by $s(g) + \text{best}(M \setminus g)$, so a group is only chosen if what remains after removing its words can still be partitioned well. A group that scores highly on its own can therefore be rejected, and a group scoring less can be preferred, if the rest of the board resolves better that way. Whether this changes the outcome in practice is reported in Chapter 5.

4.3 Solving under the Interactive Setting

The interactive setting allows only one group to be submitted at a time, so the solver must select the group it currently believes in the most. This choice cannot be made from a group's own score alone. As established in Section 4.2, a group should be picked when the remaining words can still form valid groups, so belief has to attach to whole partitions rather than to individual groups. Figure 4.3 gives an overview of the resulting loop.

A probability for every partition requires the score of every partition, not just the highest one. The recurrence used in Section 4.2 does not provide this. At each step it keeps only the best score reached so far and discards every other choice along the way. Under the interactive setting, every partition's score is instead computed directly. A partition's

score is the sum of its four group scores, looked up and added for all 2,627,625 partitions at once.

Thus, in this thesis, a group's belief is obtained by summing the probability of every partition that contains it. In order to do so, the solver needs to hold the probability for every partition. The probability distribution is transformed from the score of each partition by using softmax.

Let h range over the $|\mathcal{H}| = 2{,}627{,}625$ partitions established in Section 3.3. Before any feedback has been received, the probability assigned to partition h is:

$$P_0(h) = \frac{\exp\big(\text{score}(h)/T\big)}{\sum_{h'} \exp\big(\text{score}(h')/T\big)}. \tag{4.3}$$

The subscript 0 marks this as the belief held before any response has been observed. The choice of temperature T affects the resulting distribution. The specific value used is determined experimentally and is discussed in Chapter 5.

Each candidate group's marginal probability is obtained by summing the probabilities of every partition that contains it. Only groups formed from words that remain unresolved are valid candidates. The group submitted each round is the one with the highest marginal probability among these candidates.

Once a group has been submitted and a response received, these probabilities are revised. The partition that contradict from the response are removed from the candidates. After that, the probabilities of the remaining partitions no longer add up to one. A normalization step is carried out, the possibilities of the remaining are scaled up by the same factor until they add up to one. This is an exact Bayesian update, made simple by the fact that responses are deterministic. From the remaining candidates, the new submission will be chosen.

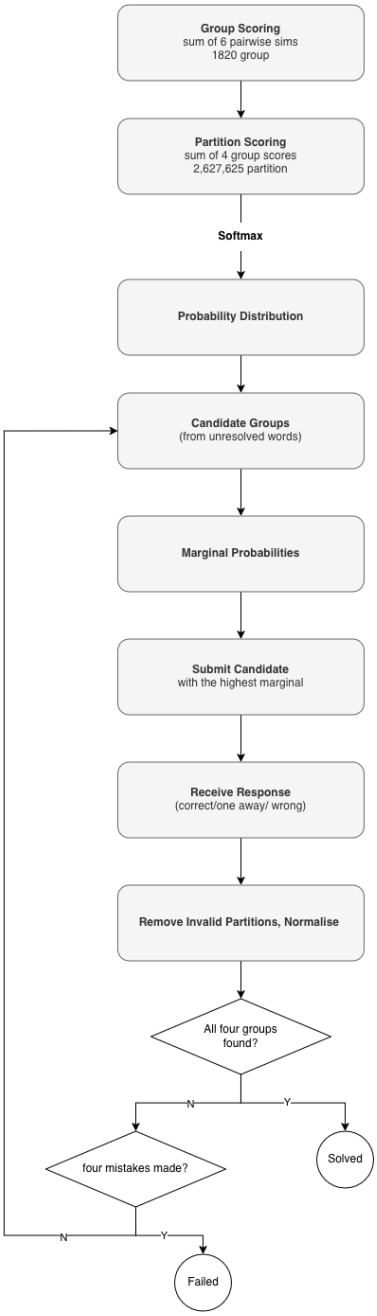

Figure 4.3: Pipeline for solving Connections under the interactive setting.

5 Experiment Setup

Bibliography

[1] *Solving Wordle Using Information Theory*. Feb. 6, 2022. URL: https://www.youtube.com/watch?v=v68zYyaEmEA (visited on 07/30/2026).

[2] Graham Todd et al. "Missed Connections: Lateral Thinking Puzzles for Large Language Models". In: *2024 IEEE Conference on Games (CoG)*. 2024 IEEE Conference on Games (CoG). Aug. 2024, pp. 1–8. DOI: 10.1109/CoG60054.2024.10645557. URL: https://ieeexplore.ieee.org/document/10645557 (visited on 01/05/2026).

[3] Prisha Samadarshi et al. *Connecting the Dots: Evaluating Abstract Reasoning Capabilities of LLMs Using the New York Times Connections Word Game*. Oct. 14, 2024. DOI: 10.48550/arXiv.2406.11012. arXiv: 2406.11012 [cs.CL]. URL: http://arxiv.org/abs/2406.11012 (visited on 07/02/2026). Pre-published.

[4] Angel Yahir Loredo Lopez, Tyler McDonald, and Ali Emami. "NYT-Connections: A Deceptively Simple Text Classification Task That Stumps System-1 Thinkers". In: *Proceedings of the 31st International Conference on Computational Linguistics*. COLING 2025. Ed. by Owen Rambow et al. Abu Dhabi, UAE: Association for Computational Linguistics, Jan. 2025, pp. 1952–1963. URL: https://aclanthology.org/2025.coling-main.134/ (visited on 12/17/2025).

[5] Emily Zhang, Yanan Jiang, and Peixuan Ye. *Learning Semantic Complexities of NYT Connections*. URL: https://web.stanford.edu/class/cs224n/final-reports/256847963.pdf.

[6] *Constraint Satisfaction Problem*. In: *Wikipedia*. Jan. 21, 2026. URL: https://en.wikipedia.org/w/index.php?title=Constraint_satisfaction_problem&oldid=1334133666 (visited on 07/31/2026).

[7] Sally C. Brailsford, Chris N. Potts, and Barbara M. Smith. "Constraint Satisfaction Problems: Algorithms and Applications". In: *European Journal of Operational Research* 119.3 (Dec. 16, 1999), pp. 557–581. ISSN: 0377-2217. DOI: 10.1016/S0377-2217(98)00364-6. URL: https://www.sciencedirect.com/science/article/pii/S0377221798003646 (visited on 07/31/2026).

[8] José Enrique Gallardo, Carlos Cotta, and Antonio José Fernández. "Solving Weighted Constraint Satisfaction Problems with Memetic/Exact Hybrid Algorithms". In: *Journal of Artificial Intelligence Research* 35 (July 28, 2009), pp. 533–555. ISSN: 1076-9757. DOI: 10.1613/jair.2770. arXiv: 1401.3477 [cs.AI]. URL: http://arxiv.org/abs/1401.3477 (visited on 07/31/2026).

[9] Matthew L. Ginsberg. *Dr.Fill: Crosswords and an Implemented Solver for Singly Weighted CSPs*. Jan. 18, 2014. DOI: 10.1613/jair.3437. arXiv: 1401.4597 [cs.AI]. URL: http://arxiv.org/abs/1401.4597 (visited on 07/31/2026). Pre-published.

[10] *Partition of a Set*. In: *Wikipedia*. May 5, 2026. URL: https://en.wikipedia.org/w/index.php?title=Partition_of_a_set&oldid=1352723986 (visited on 07/31/2026).

[11] Tobias Müller. "Solving Set Partitioning Problems with Constraint Programming". In: *Proceedings of the Sixth International Conference on the Practical Application of Prolog and the Fourth International Conference on the Practical Application of Constraint Technology (PAPPACT98)*. London, UK: The Practical Application Company Ltd., 1998, pp. 313–332. URL: https://www.ps.uni-saarland.de/Publications/documents/Mueller_98a.pdf (visited on 07/31/2026).

[12] *Information Theory*. In: *Wikipedia*. July 26, 2026. URL: https://en.wikipedia.org/w/index.php?title=Information_theory&oldid=1366101152 (visited on 08/02/2026).

[13] C. E. Shannon. "A Mathematical Theory of Communication". In: *The Bell System Technical Journal* 27.3 (July 1948), pp. 379–423. ISSN: 0005-8580. DOI: 10.1002/j.1538-7305.1948.tb01338.x. URL: https://ieeexplore.ieee.org/document/6773024 (visited on 08/02/2026).

[14] Henry Pinkard and Laura Waller. *A Visual Introduction to Information Theory*. Mar. 6, 2026. DOI: 10.48550/arXiv.2206.07867. arXiv: 2206.07867 [cs.IT]. URL: http://arxiv.org/abs/2206.07867 (visited on 08/02/2026). Pre-published.

[15] Talal Aladaileh et al. "Solving Wordle Using Information Theory". In: *Northeast Journal of Complex Systems (NEJCS)* 8.1 (Apr. 6, 2026). ISSN: 2577-8439. DOI:

10.63562/2577-8439.1146. URL: https://orb.binghamton.edu/nejcs/vol8/iss1/6.

[16] Michael Cunanan and Michael Thielscher. *On Optimal Strategies for Wordle and General Guessing Games*. May 16, 2023. DOI: 10.48550/arXiv.2305.09111. arXiv: 2305.09111 [cs.AI]. URL: http://arxiv.org/abs/2305.09111 (visited on 08/05/2026). Pre-published.

[17] Donald E. Knuth. "The Computer as Master Mind". In: *Journal of Recreational Mathematics* 9.1 (1976), pp. 1–6. URL: http://colorcode.laebisch.com/links/Donald.E.Knuth.pdf (visited on 08/06/2026).

[18] *Connections - Group Words That Share a Common Thread*. URL: https://www.nytimes.com/games/connections (visited on 08/08/2026).

[19] *Connections - Group Words That Share a Common Thread*. URL: https://www.nytimes.com/games/connections (visited on 08/08/2026).

[20] *Connections - Group Words That Share a Common Thread*. URL: https://www.nytimes.com/games/connections (visited on 08/08/2026).

[21] *Connections - Group Words That Share a Common Thread*. URL: https://www.nytimes.com/games/connections (visited on 08/08/2026).

[22] Jose Camacho-Collados and Mohammad Taher Pilehvar. *From Word to Sense Embeddings: A Survey on Vector Representations of Meaning*. Oct. 26, 2018. DOI: 10.48550/arXiv.1805.04032. arXiv: 1805.04032 [cs.CL]. URL: http://arxiv.org/abs/1805.04032 (visited on 08/20/2026). Pre-published.

[23] *Dynamic Programming*. In: *Wikipedia*. July 20, 2026. URL: https://en.wikipedia.org/w/index.php?title=Dynamic_programming&oldid=1365153484 (visited on 08/21/2026).

[24] Michael Held and Richard M. Karp. "A Dynamic Programming Approach to Sequencing Problems". In: *Journal of the Society for Industrial and Applied Mathematics* 10.1 (Mar. 1962), pp. 196–210. DOI: 10.1137/0110015.